

Crypto Trading Bot — R&D Prototype

Your Engineering Action Plan & Roadmap

Prepared for: Daniel Iyalekhue — Platform Engineer, Strategy Developer & ML Lead

Prepared: Monday, 15 June 2026 (Programme Week 3 of 10)

Based on: Business Requirements Document v1.2 (15 Jun 2026), Kickoff Meeting Notes (12 Jun 2026), Project Brief — "Crypto Trading Bot"

Team: Team 1 - CTB · Blockchain Advisors Summer Internship 2026

Part 1 — Where Things Stand Right Now

1.1 The 30-second version

Five interns are building an R&D prototype of a multi-venue crypto trading bot over an 8-week, 8h/week-per-person programme (1 June – 4 August 2026). The goal is **learning and a reusable platform — not profit**. The bot will connect to a decentralized exchange (DEX) chosen through a research process, run three competing strategies (rule-based, technical-analysis, and machine-learning) through one shared interface, and only ever risk a tiny, hard-capped amount of real money. Almost all testing happens through backtesting (replaying historical data) and paper trading (simulated trades on live data, no real money).

You are the only engineer on the team. Everyone else is doing research, defining risk rules, tracking budgets, or writing up findings — but the actual platform (repo, infrastructure, data pipeline, strategy framework, backtester, risk engine, and the ML strategy) is built by you.

1.2 Your ownership map

Epic	Name	Owner	Is this you?
0	Venue & Pair Selection	B M Rahidul Haq Bishal (research/risk lead)	No — but it gates Epics 2–4
1	Foundation & Skeleton Architecture	Daniel	Yes
2	Market Data & Paper Trading	Daniel	Yes
3	Strategy Framework & Backtesting	Daniel	Yes
4	Risk Management & First Live Trades	Daniel builds it; Bishal defines risk numbers, Affan approves capital cap	Yes (the engine)
5	Observability & Reporting	Dashboard = Daniel ; analysis/writeup = Khurram/Liam/Affan	Partially
6	ML Serving (stretch)	Daniel	Yes — first thing cut if behind schedule

In short: of the 7 epics, you build the engineering core of 5 of them, plus the dashboard for a 6th. Everyone else's work either feeds inputs into your build (risk parameters, venue choice) or consumes its outputs (reports, findings).

1.3 What changed at the last minute (v1.2)

PM change: Ibraheem Akram has moved to a different project (Sigalmark Market Discovery). Muhammad F. Khurram is now PM for this team **on a placeholder basis** — there has been no contact from him since the first weekly meeting, so his involvement is unconfirmed.

Practical implication for you: don't block on Jira tickets that may not arrive. The BRD (Section 6) is detailed enough — user stories and acceptance criteria for Epics 1–3 — to start directly from. If no tickets exist by mid-week, raise it in the *Team 1 - CTB* channel so it doesn't quietly cost you a week.

1.4 Where "today" sits on the timeline

- Programme window: **1 June – 4 August 2026** (10 calendar weeks)
 - Today, **Monday 15 June**, is the start of **Week 3** — what the BRD calls **Phase 1**
 - Phase 1 (this week only) = Epic 0 research (Bishal, in parallel) + **Epic 1 — your repo/Docker/CI skeleton**
 - ~7 weeks remain after this week
-

Part 2 — The Whole Roadmap, Through Your Eyes

This is the full 10-week programme, mapped to calendar weeks (the programme started Monday 1 June 2026) and filtered to what matters for you.

Week	Dates	Phase	What's happening team-wide	Your focus
W1–W2	1–14 Jun	—	Kickoff, BRD drafted	—
W3	15–21 Jun	1	Epic 0 venue/pair research runs in parallel (Bishal)	Epic 1: repo, Docker Compose, secrets, CI. Resolve TimescaleDB/Redis decision — due now.
W4	22–28 Jun	2	Epic 0 gate/approval expected end of W3 / start of W4. Liam on leave 19–26 Jun.	Epic 2 starts: DEX data ingestion against the chosen venue, paper-trading simulator, feed-health observability.
W5	29 Jun–5 Jul	2	—	Finish Epic 2.
W6	6–12 Jul	3	—	Epic 3 starts: pluggable strategy interface, rule-based + TA strategies.
W7	13–19 Jul	3	Live capital cap must be approved by end of this week (Affan/sponsor).	Finish Epic 3: backtester, baseline strategies, comparison reports.
W8	20–26 Jul	4	—	Epic 4: risk manager, kill switch, pre-trade checks, first tiny live trades.
W9	27 Jul–2 Aug	5–6	—	Epic 5: dashboard/reporting support. Epic 6 (ML, stretch) only if on schedule.
W10	3–4 Aug	5–6	Handover 4 Aug	Wrap-up, documentation, final findings.

Note: Epic 6 (ML strategy) is explicitly flagged in the BRD as the first thing to descope if Epics 1–4 slip. Don't let ML pull focus until the core platform (data → strategy → risk → execution loop) is solid.

Part 3 — This Week (W3): Your To-Do List

3.1 Action 1 — Close out the TimescaleDB / Redis decision (do this first, today/tomorrow)

This is action item #5 from the kickoff meeting: "Decide TimescaleDB and Redis infrastructure before Epic 1." It sounds bigger than it is — the BRD has already made most of this decision; you mainly need to confirm the details and write them down so the action item is closed.

What these things actually are (plain English)

- **TimescaleDB** = PostgreSQL (a normal relational database) plus an extension that's optimised for time-series data. You'll use it to store market prices/candles, order book snapshots, and the state of orders/positions/config. Think of it as **the bot's memory**.
- **Redis** = an in-memory data store used here as an **event bus** — a fast messaging layer so that, say, the market-data component can broadcast "new price tick" and the strategy engine reacts instantly, without the two components needing to know about each other directly. Think of it as **the bot's nervous system**.

The decision (recommended)

1. **Event bus: Redis**, not NATS. The BRD lists "Redis / NATS" as options — Redis is simpler to operate, has a mature Python client (`redis-py`), and easily handles the >1000 events/sec performance target at this scale. NATS adds operational complexity with no benefit here.
2. **Both run as Docker containers**, defined in `docker-compose.yml` alongside your app skeleton — this is also literally what Epic 1's acceptance criteria (E1-US2) requires:
 - `timescale/timescaledb:latest-pg16` — official TimescaleDB image (bundles Postgres 16)
 - `redis:7-alpine` — official, lightweight Redis image
3. **Where it runs for now:** locally, on your machine, via Docker Desktop — fine for development, backtesting, and paper trading.

Open question to raise this week: BRD §10 assumes "a self-hosted Linux VPS or homelab is available for always-on components." From Epic 2 onward, market-data ingestion needs to run continuously (24/7) — your laptop can't do that. If nobody has actually provisioned a VPS/homelab yet, flag this **now** in the Team 1 - CTB channel. Setting one up has lead time, and it'll block you in ~2 weeks if it's not sorted.

How to close the action item

Write a short note (4–6 sentences) covering: the tech choice (Redis over NATS, TimescaleDB via Docker), the exact images/versions, where it runs for now (local Docker Compose), and the open VPS/homelab question. Post it in the Team 1 - CTB channel (and add it to your repo's README or a short `docs/decisions/` note once the repo exists). That's the action item closed.

3.2 Action 2 — Epic 1: Foundation & Skeleton Architecture (rest of this week)

Goal: "Repo, config, secrets, CI, and skeleton architecture in place" — i.e. one command (`docker compose up`) brings up your database, Redis, and an empty app shell; no secrets ever touch git; and a CI pipeline runs basic checks on every pull request.

Story	What it means	Done when...
E1-US1	Monorepo with clear module boundaries	Repo has folders: <code>data/</code> , <code>engine/</code> , <code>execution/</code> , <code>risk/</code> , <code>strategies/</code> , <code>backtest/</code> , <code>observability/</code> , <code>infra/</code> . README explains each one.
E1-US2	Docker Compose local setup	<code>docker compose up</code> brings up TimescaleDB, Redis, and an app skeleton container with no errors.
E1-US3	Secrets handled out-of-band	<code>.env.example</code> exists with placeholders; real <code>.env</code> is git-ignored; a pre-commit hook blocks accidental secret commits.
E1-US4	CI pipeline (lint, type-check, tests)	GitHub Actions runs on every PR: <code>black / flake8</code> , <code>mypy</code> , <code>pytest</code> with >80% coverage on core modules.

Full step-by-step instructions for all four stories are in Part 5.

Part 4 — Docker & Infra Crash Course (you said this is new — start here)

4.1 The four words that matter

- **Image** — a frozen snapshot of an app + everything it needs to run (like a ZIP file that contains a tiny computer). E.g. "redis:7-alpine" is a ready-made image someone else built for you.
- **Container** — a running instance of an image. You can start, stop, and delete containers without affecting the image they came from.
- **Dockerfile** — a recipe for *building your own* image (used for your app skeleton, since nobody published "Daniel's trading bot" as an image).
- **docker-compose.yml** — a file that describes a *group* of containers (your database, Redis, and app) and how they connect, so you can start everything with one command.

4.2 Why this project uses Docker

The BRD requires "Docker Compose on Linux VPS or self-hosted homelab" and "each application sandboxed in its own container." Practically, this means: TimescaleDB, Redis, and your Python app each run in their own isolated box, all wired together by `docker-compose.yml`. This is also exactly why your "decide infra" action item and Epic 1 overlap — deciding the infra *is* writing the Compose file.

4.3 Install Docker Desktop (Mac)

1. Download **Docker Desktop for Mac** from Docker's official site (search "Docker Desktop download" — pick the build matching your Mac's chip, Apple Silicon or Intel).
2. Install it like any other Mac app, then open it once — it needs to finish starting before `docker` commands work.
3. Verify in Terminal:

```
docker --version
docker compose version
```

Both should print version numbers.

4.4 Command cheat-sheet (the ~6 you'll actually use)

Command	What it does
<code>docker compose up -d</code>	Start everything in the Compose file, in the background
<code>docker compose down</code>	Stop and remove all containers (data in volumes survives)
<code>docker compose ps</code>	List running containers and their status
<code>docker compose logs -f app</code>	Stream logs from the "app" service (Ctrl+C to stop watching)

<code>docker compose exec app bash</code>	Open a shell <i>inside</i> the running app container
<code>docker compose build</code>	Rebuild your app image after changing the Dockerfile/code

Part 5 — Step-by-Step: Building Epic 1

Work through these in order. Each step maps back to one of the four E1 user stories from Part 3.2.

Step 1 — Create the repo (E1-US1)

Create a new GitHub repository (e.g. `crypto-trading-bot`), clone it locally, and create this folder structure:

```
crypto-trading-bot/
├── data/           # market data ingestion (CCXT, web3/RPC clients, MarketState models)
├── engine/         # strategy engine – orchestration, lifecycle, state
├── execution/      # order routing / venue abstraction (DEX tx today, CEX later)
├── risk/           # risk manager, kill switch, position limits
├── strategies/     # pluggable strategy plugins (rule-based, TA, ML)
├── backtest/       # backtesting harness + paper-trading simulator
├── observability/  # logging, metrics, dashboard
├── infra/          # Dockerfiles, deployment configs
├── tests/          # unit + integration tests
├── docker-compose.yml
├── .env.example
├── .gitignore
├── .pre-commit-config.yaml
├── requirements.txt
└── README.md
```

In the README, write one or two sentences per folder explaining its responsibility — this *is* the acceptance criterion ("README explains boundaries") and doubles as documentation for the bus-factor risk the BRD calls out (you're the only engineer — write things down as you go).

Step 2 — docker-compose.yml: TimescaleDB + Redis + app skeleton (E1-US2)

```
version: "3.9"

services:
  timescaledb:
    image: timescale/timescaledb:latest-pg16
    container_name: ctb-timescaledb
    restart: unless-stopped
    environment:
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
      POSTGRES_DB: ${POSTGRES_DB}
    ports:
      - "5432:5432"
    volumes:
      - timescale_data:/var/lib/postgresql/data

  redis:
    image: redis:7-alpine
    container_name: ctb-redis
    restart: unless-stopped
    ports:
      - "6379:6379"
    volumes:
      - redis_data:/data

  app:
    build:
      context: .
      dockerfile: infra/Dockerfile
    container_name: ctb-app
    depends_on:
      - timescaledb
      - redis
    env_file:
      - .env
    volumes:
      - ./app

volumes:
  timescale_data:
  redis_data:
```

infra/Dockerfile (your app skeleton)

```
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

CMD ["python", "-m", "engine.main"]
```

engine/main.py (proves the skeleton actually works)

```
import os
import redis
import psycopg2

def main():
    r = redis.Redis(host=os.environ["REDIS_HOST"], port=int(os.environ["REDIS_PORT"]))
    r.ping()
    print("Redis OK")

    conn = psycopg2.connect(
        host=os.environ["POSTGRES_HOST"],
        port=os.environ["POSTGRES_PORT"],
        user=os.environ["POSTGRES_USER"],
        password=os.environ["POSTGRES_PASSWORD"],
        dbname=os.environ["POSTGRES_DB"],
    )
    print("Postgres OK")
    conn.close()

if __name__ == "__main__":
    main()
```

requirements.txt (starting point)

```
redis
psycopg2-binary
python-dotenv
```

Test it: run `docker compose up -d`, then `docker compose logs app`. You should see "Redis OK" and "Postgres OK" with no errors. That's E1-US2 done.

Step 3 — Secrets handling (E1-US3)

.env.example (commit this — it's just placeholders)

```
# Database
POSTGRES_USER=ctb
POSTGRES_PASSWORD=changeme
POSTGRES_DB=ctb
POSTGRES_HOST=timescaledb
POSTGRES_PORT=5432

# Redis
REDIS_HOST=redis
REDIS_PORT=6379

# DeFi RPC – fill in once Epic 0 selects a chain
RPC_URL=

# Live trading switch – leave false until Epic 4
LIVE_ENABLED=false
```

Copy this to a real `.env` file locally and fill in real values. **Never commit** `.env`.

.gitignore additions

```
.env
*.pyc
__pycache__/
.venv/
*.log
```

.pre-commit-config.yaml — blocks secrets and enforces style

```
repos:
- repo: https://github.com/pre-commit/pre-commit-hooks
  rev: v4.6.0
  hooks:
    - id: check-merge-conflict
    - id: end-of-file-fixer
    - id: trailing-whitespace

- repo: https://github.com/psf/black
  rev: 24.4.2
  hooks:
    - id: black

- repo: https://github.com/PyCQA/flake8
  rev: 7.0.0
  hooks:
    - id: flake8

- repo: https://github.com/gitleaks/gitleaks
  rev: v8.18.4
  hooks:
    - id: gitleaks
```

Install with `pip install pre-commit && pre-commit install` — now every `git commit` runs these checks automatically, including a scan for accidentally-committed secrets (the gitleaks hook).

Step 4 — CI pipeline (E1-US4)

`.github/workflows/ci.yml`

```
name: CI

on:
  pull_request:
  push:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.11"
      - run: pip install -r requirements.txt -r requirements-dev.txt
      - run: black --check .
      - run: flake8 .
      - run: mypy .
      - run: pytest --cov=. --cov-report=term-missing
```

You'll need a small `requirements-dev.txt` with `black`, `flake8`, `mypy`, `pytest`, `pytest-cov`. The >80% coverage target only needs to apply to modules you've actually written — an empty skeleton with one tested function is fine for now; coverage grows with the codebase.

Step 5 — Verify against the acceptance criteria

- ☐ Repo has all 8 module folders + README explaining each (E1-US1)
 - ☐ `docker compose up` brings up DB + Redis + app with no errors (E1-US2)
 - ☐ `.env` is git-ignored, `.env.example` committed, pre-commit blocks secrets (E1-US3)
 - ☐ CI runs lint + type-check + tests on every PR (E1-US4)
 - ☐ Infra decision (Redis + TimescaleDB via Docker, local for now) posted to the team channel
-

Part 6 — What's Coming Next (so you know the shape of the road)

Epic 2 — Market Data & Paper Trading (W4–W5)

Once Epic 0 picks a chain/DEX, you'll connect to it via RPC (web3.py or ethers) to read pool prices/reserves, normalise them into a shared `MarketState` format, and persist to TimescaleDB. Alongside that, you'll build a paper-trading simulator (realistic fills/fees/slippage, no real orders) and a feed-health check (alert if data goes stale >30s). *This is gated by Epic 0 — don't start the RPC work until the venue is approved, but the paper simulator and MarketState schema can be designed in the meantime.*

Epic 3 — Strategy Framework & Backtesting (W6–W7)

Define one common interface (`receive(market_state) -> Optional[Orders]`), then build three strategies behind it: rule-based (price-range orders), TA-signal (MA cross / RSI / Bollinger), and ML-driven. Build a backtester that runs the *same* strategy code against historical data, plus baseline strategies (buy-and-hold/no-op) for comparison (Sharpe, max drawdown, win rate).

Epic 4 — Risk Management & First Live Trades (W8)

Build the risk manager: hard capital cap, pre-trade checks (position size, exposure, drawdown, daily loss), a global kill switch (<1s activation), and state reconciliation on startup. Live trading is gated behind an explicit `LIVE_ENABLED` flag + manual confirmation. Bishal supplies the actual risk numbers (caps, drawdown limits); Affan/sponsor must approve the capital cap by end of W7 before this epic can go live.

Epic 5 — Observability & Reporting (W9)

Your part: a browser-based dashboard showing positions, P&L, signals, and fills, auto-refreshing under 5s, plus structured JSON logs with trace IDs. Prometheus/Grafana were explicitly deferred as "too heavy for this stage" — a lightweight custom dashboard is fine. The written findings document is mostly Khurram/Affan's job, fed by your reports.

Epic 6 — ML Serving (W9–W10, stretch)

A model-serving container on local GPU, separate from the trading loop, exposed via REST/gRPC. The ML strategy plugs into the same `Strategy` interface as everything else, so it automatically passes through the risk manager. **Only start this if Epics 1–4 are done** — it's the designated first cut if the schedule slips.

Part 7 — Things to Flag or Confirm With the Team This Week

Topic	Why it matters	Who to ask
VPS / homelab access	Epic 2 onward needs 24/7 data ingestion — your laptop can't provide that. Lead time risk if not yet provisioned.	Laurentiu Nae / sponsor (Affan)
PM situation	Khurram (placeholder PM) unconfirmed since first meeting — Jira tickets may not appear. Don't block on them.	Laurentiu Nae
Redis vs NATS	This plan recommends Redis for simplicity. Confirm nobody on the team has a strong reason to use NATS instead before you build around it.	Team 1 - CTB channel
Epic 0 gate timing	You need to know exactly when the venue/pair selection is approved so you know when Epic 2's RPC work can start.	Bishal (B M Rahidul Haq Bishal)

Part 8 — Working Solo: Habits That Will Save You

The BRD explicitly calls out "single-engineer dependency" as a **Medium likelihood / Critical impact** risk, mitigated by "keep architecture simple; document as built." A few habits make this painless rather than a burden:

- **Write decisions down as you make them.** A short `docs/decisions/` folder with one file per significant choice (e.g. "0001-event-bus-redis-vs-nats.md") takes two minutes and saves hours of re-explaining later.
 - **Keep the README current.** If a teammate (or future-you) needs to understand the system, the README + folder structure should be enough to get oriented without asking you.
 - **Small, frequent commits with CI green.** Even reviewing your own PRs is useful — it forces you to read your diff with fresh eyes, and CI catches mistakes before they compound.
 - **A short weekly written status update** (3-4 bullets: done / in progress / blocked / decisions made) posted to the Team 1 - CTB channel. This is your insurance against the bus-factor risk and keeps the PM situation from becoming a problem.
-

Appendix — Glossary

Term	Definition
CEX	Centralized Exchange (e.g. Binance, Kraken) — out of scope for now, stretch only
DEX	Decentralized Exchange / AMM (e.g. Uniswap) — this project's primary venue type
AMM	Automated Market Maker — the pricing mechanism behind most DEXs (e.g. Uniswap pools)
TA	Technical Analysis — indicator-driven trading (moving averages, RSI, Bollinger Bands)
ML	Machine Learning
P&L	Profit and Loss
MEV	Maximal Extractable Value — front-running/sandwiching risk on-chain
Kill Switch	Emergency halt that stops all trading activity and cancels open orders
Paper Trading	Simulated trading on live market data with no real capital at risk
RPC	Remote Procedure Call — how your code talks to a blockchain node to read prices/state
Event Bus	A messaging layer (here: Redis) that lets components communicate without direct coupling
Docker Compose	A tool for defining and running multi-container applications with one command

This document was generated from: BRD - Crypto Trading Bot.md (v1.2, 15 Jun 2026), Team 1 - CTB kickoff meeting notes (12 Jun 2026), and the "Crypto Trading Bot" entry in the Summer 2026 Internship project brief.